Efficient instruction-level parallelism analysis using tropical semiring 'matrix multiplication'.

Bryan Abi Karam

October 9, 2025

1 Introduction

Instruction-level parallelism (ILP) is a technique commonly used by high-performance processors to improve performance. Previous studies [3] have investigated the theoretical limits of ILP in programs using models of their dynamic execution.

An upper bound for parallelism in a program arises from its critical path: the longest chain of dependent instructions in the program's dynamic execution. For a machine that takes at least one cycle per instruction, the program cannot be run faster than the time it takes to run the critical path sequentially.

Therefore an upper bound for ILP in a program can be calculated as the number of dynamic instructions in the program's execution divided by the length of the critical path.

The dependencies of instructions in a program execution can be represented by its full dynamic dependency graph (DDG) [1], which is a directed acyclic graph with one node per dynamically executed instruction in a program. The edges of the graph represent dependencies between dynamically executed instructions, and so the length of the critical path is the length of the longest path through the program's DDG.

Tropical analysis is a branch of pure mathematics using the tropical semiring [2], also known as the min/max-plus algebra, which is the semiring

$$(\mathbb{R} \cup \{+\infty\}, \oplus, \otimes)$$

with operations

$$a \oplus b = \min(a, b) \text{ or } \max(a, b)$$
$$a \otimes b = a + b$$

The problem of finding the longest path in a DAG can be expressed as a series of 'matrix multiplications' in the max tropical semiring, as shown by Section I-B of Chadwick [2].

In this project I will create a library for performing such matrix calculations efficiently, considering aspects such as vectorisation, tiling, and representation sparsity. I will work with CPU based compute and may work with GPU based compute as an extension.

This introduction is adapted from the abstract of Chadwick [2].

2 The project

2.1 Project core

First, a CPU-based library for performing individual matrix computations in the tropical semiring should be implemented, using a dense representation of matrices. Techniques such as vectorisation and tiling should be considered for performance gains.

Matrix multiplication expressions will be generated from a program execution's dependency structure as a graph, with one node per register per dynamic instruction, in the column-based representation proposed in Section I-B of Chadwick [2].

The aforementioned matrix computation library will be used to compute the results of these expressions, using techniques such as caching and evaluation order optimisation to maximise efficiency.

Finally, a benchmark suite for testing performance of implementations will be implemented, and a baseline performance will be established using a 'classical' technique for computing dependency chains (i.e. simulating one instruction at a time naively, computing its dependencies one-by-one).

2.2 Project extensions

2.2.1 Sparse matrix representation

The core of the project implements a matrix computation library using a dense representation of matrices. In an effort to improve memory usage, and potentially improve performance (as the cost of sparse multiplication is proportional to the number of non-zeroes in the matrix, rather than just the number of rows and columns), I will implement a similar CPU-based matrix computation library using a sparse representation of matrices as an extension to the project core.

The sparse library will be benchmarked against the dense library for register-only program execution dependency graphs.

2.2.2 GPU compute

The core of the project implements a CPU-based matrix computation library. To exploit the excellent parallel power of GPUs, I will implement two GPU-based matrix computation libraries, for dense and sparse representations of matrices, using a programming model such as OpenCL or CUDA.

The GPU libraries will be benchmarked against their CPU counterparts for register-only program execution dependency graphs.

2.2.3 Through-memory dependencies

So far we have only computed results of matrix expressions generated from register-only program execution dependency graphs. However, for real programs, the true critical path often involves memory operations.

A simple approach is to naively include memory locations as graph nodes alongside registers for each dynamic instruction, but this results in a catastrophic explosion of matrix sizes. Nevertheless, these matrices are extremely sparse, as most memory operations access only two or three locations at a time, and so are usable with sparse representations. For this approach, I will benchmark the performance of the two previously implemented sparse matrix computation libraries (running on CPU and GPU) against the baseline.

An alternative approach is to model the effects of memory operations using tropical semiring *addition* rather than just multiplication, as described in Section II-B of Chadwick [2]. For this approach, I will benchmark the performance of the two previously implemented dense matrix computation libraries (running on CPU and GPU) against the baseline.

3 Starting point

Existing standard libraries for matrix multiplication cannot be used as they are specialised for linear algebra (as opposed to tropical algebra). Additionally, techniques such as Strassen's algorithm do not apply in a semiring.

A suite of tools will be provided by Alexandra Chadwick, the project supervisor, to generate matrix expressions from program executions. The tools will do the following:

- Compute the static dependencies of every static instruction in a program.
- Turn the above into a matrix per instruction.
- Dump the dynamic flow of execution of a program.

- Use the dynamic flow and the per-instruction matrices to create a matrix expression.

Generated matrix expressions will be used as input to the matrix computation libraries.

4 Success criteria and evaluation plan

4.1 Success criteria

For the dissertation to be considered a success, all items in the project core (Section 2.1) should be completed, specifically the following items:

- Implement a CPU-based library for performing individual matrix computations in the tropical semiring, using a dense representations of matrices.
- Use the CPU-based library to compute the result of large matrix expressions for computing longest paths in register-only DDGs.
- Establish a baseline performance using a classical technique for computing dependency chains.
- Implement a benchmark suite for testing performance of implementations.

For the core to be considered a success, the CPU-based library should demonstrate a performance improvement over the naive baseline for computing the length of the critical path.

If time permits, I will complete the three project extensions described in Section 2.2, prioritising them in the order they are presented. The success criteria for each extension is as follows:

- Sparse matrix representation: The implementation from this extension should demonstrate a reduction in memory usage and an improvement in performance for very sparse matrices over the CPU-based dense implementation from the project core.
- GPU compute: The implementations from this extension should show performance improvement over their CPU-based counterparts.
- Through-memory dependencies: The benchmarks from this extension should demonstrate a performance improvement for the matrix computation libraries over the naive baseline for computing the length of the critical path in a full DDG.

All implementations of the matrix computation library should evaluate matrix expressions **correctly**.

4.2 Evaluation plan

The evaluation of the matrix computation libraries will be conducted using a comprehensive set of benchmarks on both register-only DDGs and ones with through-memory dependencies (addition and sparse representations). All implementations will be benchmarked against the naive baseline (sequential dependency chain computation) to quantify performance improvements.

For each benchmark, the following metrics will be collected:

- **Execution time**: Total time taken to compute the longest path, or evaluate the matrix expression.
- **Memory usage**: Peak and average memory consumption during computation.
- **Correctness**: Verification that computed results match expected results for known cases.

Results will be analysed to determine which approach is most effective for each situation, and to validate the success criteria detailed above.

5 Plan of work

10/10-24/10: Implement naive baseline

Doing this before working with matrix computations will allow me to understand the starting state of dependency chain computation.

Deliverable: Naive baseline.

25/10-7/11: Implement CPU-based dense matrix computation library

I will first implement the library for indiviual matrix multiplications, focusing on performance optimisations.

Deliverable: Individual matrix computation library implemented.

8/11-21/11: Use CPU-based library to compute matrix expressions

I will use the aforementioned library in computing the results of large matrix expressions. Work from this sprint may spill over into the next sprint due to an assessment deadline for the Category Theory module I am taking.

Deliverable: Library for computing matrix expressions implemented.

22/11-5/12: Catch up and benchmark suite

First I will complete any outstanding work from the previous sprint, and develop a benchmark suite for testing solution performance in preparation for evaluation.

Deliverable: Benchmark suite implemented.

6/12-19/12: Evaluation of core

I will evaluate the results of the project core against the success criteria using the benchmark suite developed in the previous sprint.

Deliverable: Evaluation of core completed.

20/12-2/1: Low-progress buffer

As this sprint falls over the holiday season, the amount of work done may be minimal. I will use this time as a buffer for any outstanding work to be completed.

3/1-16/1: Sparse matrix representation

This is the first of the project extensions.

Deliverable: Matrix computation library using sparse representation implemented.

17/1-30/1: GPU-based matrix computation libraries

This is the second of the project extensions. Due to an assessment deadline for the Category Theory module I am taking over the first week of Lent term, work from this sprint may spill over into the next sprint.

Deliverable: Matrix computation library using GPU compute implemented.

31/1-13/2: Catch up and progress report

I will catch up on any outstanding work, and dedicate time to preparing the progress report and presentation due on 06/02/2025.

14/2-27/2: Evaluation of first two extensions

I will evaluate the results of the first two project extensions against the success criteria.

Deliverable: Evaluation of first two extensions completed.

28/2-13/3: Investigate through-memory dependencies

The second half of this sprint will coincide with an assessment deadline for the Multicore Semantics and Programming module I am taking, and so progress may be limited.

This is the third of the project extensions. This extension is an investigation of the performance of the previously implemented libraries on full DDGs, and so may not require as much work as the previous extensions.

Deliverable: Investigation of through-memory dependencies completed.

14/3-27/3: Catch up and evaluate last extension

The first half of this sprint will coincide with an assessment deadline for the Multicore Semantics and Programming module I am taking, and so progress may be limited.

This sprint will be used to catch up on any outstanding work, and complete evaluations for all extensions.

Deliverable: Evaluation of all extensions completed.

28/3-10/4: Start write-up

In this sprint I will plan the dissertation structure, and start writing chapters 1 (introduction) and 2 (preparation) of the dissertation for review.

If time permits, I will start writing chapter 3 (implementation).

Deliverable: Dissertation document plan drafted, and chapters 1 and 2 written.

11/4-24/4: Continue write-up

I will address feedback received on chapters already written, then I will continue writing the dissertation, aiming to complete chapters 3 (implementation), 4 (evaluation), and 5 (conclusions).

Deliverable: Dissertation chapters 3, 4 and 5 written.

25/4-8/5: Submit dissertation

I will address feedback on all the wriitng done in previous sprints, and then proofread the dissertation to be ready for submission 1 week before the deadline of 15/05/2025.

6 Resource declaration

I will use my personal laptop (Intel i7-12700H, 16GB RAM, 1TB SSD) for developing the code artefact and writing the thesis. I will use the Git VCS for both the code and thesis source, as two separate repositories hosted on Github using a personal account. I will make an effort to push any changes to the remote repository as soon as they are committed, at least once a day.

In the event of my device failing, I will use university-provided machines in my college's computer room or library until I am able to acquire a new laptop.

I accept full responsibility for this machine and I have made contingency plans to protect myself against hardware and/or software failure.

References

[1] T. Austin and G. Sohi, *Dynamic dependency analysis of ordinary programs*, in [1992] Proceedings the 19th Annual International Symposium on Computer Architecture, 1992, pp. 342–351.

[2] A. Chadwick, *On tropical semirings and the limits of instruction-level parallelism.* `https://www.cl.cam.ac.uk/~awc32/tropical-ilp.pdf`, 2025.

[3] A. W. Chadwick, M. Erdős, U. Bora, A. Bhosale, B. Lytton, Y. Guo, R. Cooper, G. Gabrielli, and T. M. Jones, *The future of instruction-level parallelism (ilp)*, in 2025 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS), 2025, pp. 350–352.